

CHƯƠNG 3: LẬP TRÌNH SERVLET

Mục tiêu:

Cung cấp cho sinh viên các kiến thức và có khả năng xây dựng ứng dụng web động sử dụng Java Servlet, có khả năng xử lý yêu cầu HTTP, tương tác với cơ sở dữ liệu thay đổi nội dung trang web. API Servlet cung cấp các giao diện và lớp để phát triển ứng dụng web. Điều này bao gồm các giao diện như Servlet, ServletRequest, ServletResponse, và các lớp cụ thể như HttpServlet. Nội dung bài này cung cấp các nội dung liên quan đến các khái niệm trong một chương trình Servlet.

Nội dung:

- HttpServlet
- Mapping Servlet, Annotations
- Forward và redirect
- Session, Cookie.
- Filter, Servlet Listener
- Servlet Exception

Bài 5: Lập trình Servlet.

Servlet trong Java là một thành phần của Java EE (Enterprise Edition) được sử dụng để xử lý yêu cầu và phản hồi trên máy chủ web. Đây là một mô hình lập trình trên máy chủ cho phép phát triển ứng dụng web động. Servlet có thể tạo ra nội dung động, kết nối với cơ sở dữ liệu, và thực hiện các tác vụ máy chủ. Chúng làm việc dựa trên giao thức HTTP và được quản lý bởi một container Servlet, giúp tối ưu hóa quá trình xử lý và quản lý trạng thái truy cập của người dùng trên web. Servlet là một phần quan trọng của phát triển ứng dụng web Java.

3.1. Tổng quan về J2EE

Java 2 Platform, Enterprise Edition (J2EE) là một nền tảng phát triển và triển khai ứng dụng phức tạp, đa tầng, và có thể mở rộng dễ dàng.

- Kiến Trúc Đa Tầng: J2EE sử dụng mô hình kiến trúc đa tầng để chia ứng dụng thành các thành phần khác nhau như giao diện người dùng, logic ứng dụng, và cơ sở dữ liệu. Điều này giúp quản lý và duy trì ứng dụng một cách hiệu quả.
- Servlets và JSP (JavaServer Pages): Cho phép phát triển giao diện người dùng động và tương tác trực tiếp với máy chủ web.
- Enterprise JavaBeans (EJB): Cung cấp mô hình lập trình cho logic ứng dụng và các dịch vụ doanh nghiệp.

- JMS (Java Message Service): Định nghĩa các giao tiếp giữa các thành phần của ứng dụng thông qua các thông điệp.
- JNDI (Java Naming and Directory Interface): Cung cấp dịch vụ tìm kiếm và quản lý các đối tượng trong môi trường phân tán.
- JTA (Java Transaction API): Hỗ trợ giao dịch trong môi trường phân tán.
- JCA (Java Connector Architecture): Cho phép các ứng dụng kết nối và tương tác với các hệ thống thông tin doanh nghiệp.
- Containers: J2EE chia thành các loại container khác nhau để triển khai các thành phần. Các loại container bao gồm Servlet Container, EJB Container, và JSP Container. Mỗi container chịu trách nhiệm cho việc quản lý và cung cấp các dịch vụ cho các thành phần tương ứng.

Cơ Chế Kết Nối và Tích Hợp: J2EE hỗ trợ các cơ chế kết nối và tích hợp như JDBC (Java Database Connectivity) cho kết nối cơ sở dữ liệu, JCA cho tích hợp với các hệ thống thông tin doanh nghiệp, và JMS cho truyền thông qua các hàng đợi thông điệp.

- Bảo Mật: J2EE cung cấp cơ chế bảo mật mạnh mẽ, bao gồm quản lý quyền, xác thực và ủy quyền, để bảo vệ ứng dụng khỏi các rủi ro bảo mật.
- Tiêu Chuẩn và Giao Tiếp Dễ Dàng: J2EE tuân theo nhiều tiêu chuẩn mở, giúp ứng dụng dễ dàng tích hợp với các công nghệ khác và tương thích với nhiều hệ thống khác nhau.
- Quản Lý Trạng Thái và Tài Nguyên: J2EE hỗ trợ quản lý trạng thái và tài nguyên trong các ứng dụng phân tán, đồng thời cung cấp cơ chế sao lưu và khôi phục dữ liệu.
- Phân Tán và Tính Mở Rộng: J2EE làm cho việc phát triển và triển khai ứng dụng phân tán trở nên dễ dàng và có thể mở rộng tốt trên nhiều máy

3.2. Tổng quan Servlet

3.2.1. Servlets là gì?

Java Servlets là các chương trình chạy trên một Web server hoặc một Application server và thực hiện như là một tầng trung gian giữa một Yêu cầu từ một trình duyệt web hoặc HTTP client với các Database hoặc các ứng dụng trên HTTP server.

Sử dụng Servlets, ta có thể thu thập Input từ người sử dụng thông qua các form trên trang web, hiển thị các bản ghi (record) từ một Database hoặc từ nguồn khác, và tạo các trang web động.

Java Servlets thường có chung mục đích: là các chương trình độc lập bởi việc sử dụng Common Gateway Interface (CGI). Nhưng so với CGI, thì Servlets có các lợi thế sau:

Hiệu năng tốt hơn đáng kể.

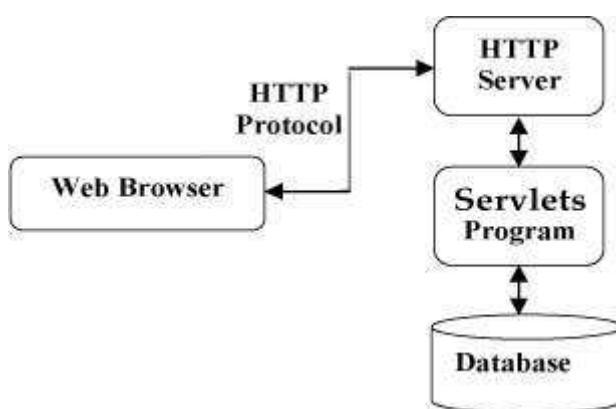
Servlets thực thi bên trong không gian địa chỉ của một Web server, không cần thiết phải tạo một tiến trình riêng biệt để xử lý mỗi yêu cầu từ Client.

Servlets là độc lập trên nền tảng bởi vì chúng được viết bằng Java. Bởi vì viết bằng Java (mà Java cung cấp tình bảo mật cao trên Server) nên Servlets là đáng tin cậy.

Tính năng đầy đủ của thư viện của các lớp trong Java là luôn luôn có sẵn cho Servlets. Nó có thể giao tiếp với Applet, Database hoặc phần mềm khác thông qua các Socket và kỹ thuật RMI mà ta đã từng thấy.

3.2.2. Cấu trúc Servlets

Cấu trúc của Servlets: Sơ đồ dưới đây minh họa cấu trúc của Servlets trong một ứng dụng web:



Hình 3: 1 Cấu trúc Servlets

Nhiệm vụ của Servlets

Servlets thực hiện các tác vụ chủ yếu sau:

Đọc dữ liệu hiển thị (explicit) được gửi bởi Client (hoặc trình duyệt) bao gồm một HTML Form trên một trang web hoặc nó cũng có thể từ một Applet hoặc một chương trình Custom từ HTTP Client.

Đọc dữ liệu yêu cầu HTTP ẩn (implicit) được gửi bởi Client (hoặc trình duyệt) bao gồm cookie, các loại media.

Xử lý dữ liệu và cho ra kết quả. Tiến trình này có thể yêu cầu Database, đang thực thi một triệu hồi tới RMI hoặc CORBA, triệu hồi một Web Service, hoặc tính toán phản hồi một cách trực tiếp.

Gửi dữ liệu hiển thị (ví dụ: tài liệu) tới các Client (hoặc trình duyệt). Tài liệu này có thể được gửi theo nhiều định dạng khác nhau, gồm text (HTML hoặc XML), nhị phân (hình ảnh GIF), Excel, .v.v.

Gửi phản hồi HTTP ẩn tới các Client (hoặc trình duyệt), thông báo cho trình duyệt hoặc Client về kiểu của tài liệu được trả về (ví dụ: HTML), thiết lập các Cookie và Caching các tham số, cùng các tác vụ khác.

Package trong Servlets

Java Servlets là các lớp trong Java chạy bởi một Web Server mà có một trình thông dịch hỗ trợ Java Servlets.

Servlets có thể được tạo bởi sử dụng các gói `javax.servlet` và `javax.servlet.http` là một phiên bản mở rộng của thư viện lớp Java để hỗ trợ các dự án phát triển có phạm vi lớn. Các lớp này triển khai Java Servlet và JSP.

Java Servlet đã được tạo và được biên dịch giống như các lớp khác trong Java. Sau khi cài đặt các gói servlet và thêm chúng vào Classpath trong máy tính được cài, ta có thể biên dịch Servlet với bộ biên dịch JDK hoặc bất kỳ bộ biên dịch nào khác.

3.2.3. Cài đặt môi trường Servlet

Giống như bất kỳ chương trình Java nào khác, ta cần biên dịch một servlet bởi sử dụng trình biên dịch `javac` của Java và sau khi biên dịch ứng dụng servlet này, nó sẽ được triển khai trong một môi trường để chạy.

Cài đặt môi trường cho Servlets bao gồm các bước sau:

- Cài đặt Java Development Kit
- Tại bước này, ta cần tải sự triển khai của Java Software Development Kit (SDK) và cài đặt biến môi trường `PATH` một cách thích hợp.
- Có thể tải SDK từ: Java SE Downloads.
- Sau khi tải xong, theo các chỉ dẫn đã cho để cài đặt và định cấu hình cho nó. Cuối cùng, thiết lập các biến môi trường `PATH` và `JAVA_HOME` để tham chiếu tới thư mục mà chứa `java` và `javac`, tương ứng là `java_install_dir/bin` và `java_install_dir`.
- Với Windows và cài đặt SDK trong `C:\jdk1.8.xxx`, sẽ đặt dòng sau trong `C:\autoexec.bat` file .

```
set PATH=C:\jdk xx\bin;%PATH%  
set JAVA_HOME=C:\jdk xx
```

Trên Windows, ta cũng có thể nhấp chuột phải vào My Computer, chọn Properties, sau đó là Advanced, và Environment Variables. Sau đó, ta cập nhật giá trị `PATH` và nhấn OK.

Trên Unix (Solaris, Linux, .v.v.), nếu cài đặt SDK trong `/usr/local/jdk1.8 xx`, và ta sử dụng C Shell, thì ta đặt dòng sau vào `.cshrc` file.

```
setenv PATH /usr/local/jdkxx/bin:$PATH  
setenv JAVA_HOME /usr/local/jdk xx
```

Nếu sử dụng một IDE (viết tắt của Integrated Development Environment) như Borland JBuilder, Eclipse, IntelliJ IDEA, hoặc Sun ONE Studio, thì ta biên dịch và thực thi một chương trình đơn giản để xác nhận rằng IDE biết nơi cài đặt Java.

3.1.4. Cài đặt Web Server: Tomcat

Một số Web server hỗ trợ các Servlet là có sẵn trên thị trường. Tomcat là một trong số các Web server miễn phí.

Apache Tomcat là một triển khai phần mềm Java Servlet và các công nghệ JavaServer Pages mã nguồn mở và có thể hoạt động như một Standalone Server để kiểm tra các servlet và có thể được tích hợp với Apache Web Server. Sau đây là các bước để cài đặt Tomcat trên máy server:

Tải phiên bản mới nhất của Tomcat từ: <http://tomcat.apache.org/>.

Mở và đặt nó vào trong vị trí thích hợp. Ví dụ: trong C:\apache-tomcat-9.0 trên Windows, hoặc trong /usr/local/apache-tomcat-9.0 trên Linux/Unix và tạo biến môi trường CATALINA_HOME chỉ tới các vị trí này.

Tomcat có thể được bắt đầu bởi thực thi lệnh sau trên các thiết bị Windows:

```
%CATALINA_HOME%\bin\startup.bat  
C:\apache-tomcat-9.0\bin\startup.bat
```

Hoặc các lệnh sau trên Unix (Solaris, Linux, .v.v.):

```
$CATALINA_HOME/bin/startup.sh  
/usr/local/apache-tomcat-9.0/bin/startup.sh
```

Sau khi cài đặt, các ứng dụng web mặc định bao gồm Tomcat sẽ là có sẵn bởi <http://localhost:8080/>. Nếu mọi thứ là hoàn hảo, thì nó sẽ hiển thị:



Hình 3: 2 Cài đặt Tomcat

Thông tin bổ sung về định hình và chạy Tomcat có thể được tìm thấy trong:
<http://tomcat.apache.org>

Tomcat có thể bị dừng bởi thực thi các lệnh sau trên thiết bị Windows:

Hoặc lệnh sau trên Unix (Solaris, Linux, .v.v.):

```
/usr/local/apache- tomcat-9.0/bin/shutdown.sh
```

Thiết lập CLASSPATH

Khi các servlet chưa là một phần của Java Platform, Standard Edition, ta phải đồng nhất các lớp servlet tới bộ biên dịch.

Nếu máy server đang chạy trên Windows, ta cần đặt các dòng sau vào trong C:\autoexec.bat file.

```
set CATALINA=C:\apache- tomcat-9.0
set CLASSPATH=%CATALINA%\common\lib\servlet-api.jar;%CLASSPATH%
```

Trên Windows, ta cũng có thể nhấp chuột phải vào My Computer, chọn Properties, sau đó là Advanced, và Environment Variables. Sau đó, cập nhật giá trị CLASSPATH và nhấn OK.

Trên Unix (Solaris, Linux, .v.v.), nếu đang sử dụng C Shell, ta đặt các dòng sau vào trong .cshrc file:

```
setenv CATALINA=/usr/local/apache- tomcat-9.0
setenv CLASSPATH $CATALINA/common/lib/servlet-api.jar:$CLASSPATH
```

Chú ý: Giả sử thư mục phát triển là C:\ServletDevel (Windows) hoặc /usr/ServletDevel (Unix) thì ta cần thêm các thư mục này trong CLASSPATH theo cách tương tự như ta đã làm ở trên.

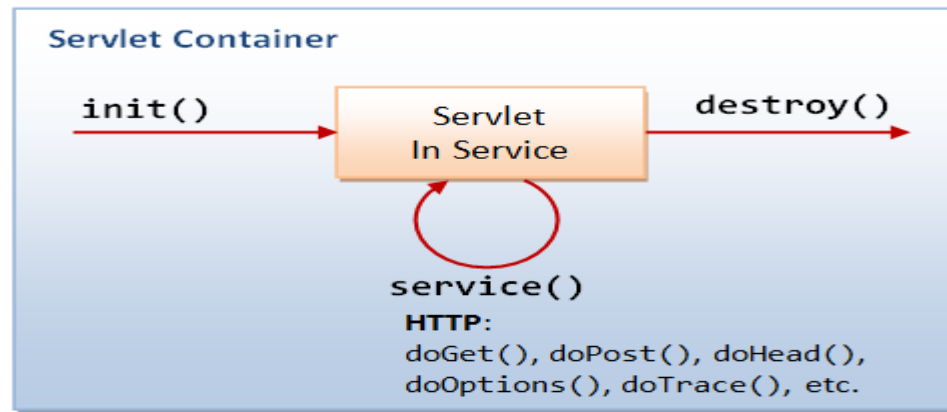
3.3. Vòng đời (Life Cycle) của Servlet

Vòng đời của Servlet có thể được định nghĩa như là một tiến trình đầy đủ từ khi được tạo ra đến khi bị hủy. Một Servlet theo các giai đoạn sau:

- Servlet được khởi tạo bởi gọi phương thức init().
- Servlet gọi phương thức service() để xử lý một yêu cầu từ Client.
- Servlet bị hủy bởi triệu hồi phương thức destroy().

Cuối cùng, servlet trở thành rác và được thu thập bởi Garbage Collector của JVM.

Tiếp theo ta đi vào chi tiết từng phương thức trong vòng đời của Servlet:



Hình 3: 3 Vòng đời của Servlet

Phương thức init()

Phương thức được thiết kế để chỉ được gọi một lần. Nó được gọi khi Servlet lần đầu được tạo, và không được gọi lại cho mỗi yêu cầu của người dùng. Vì thế, nó được sử dụng cho các khởi tạo one-time, giống như phương thức init của Applet.

Thường thì, Servlet được tạo khi một người dùng lần đầu triệu hồi một URL tương ứng với Servlet đó, nhưng ta cũng có thể xác định rằng Servlet này được tải khi Server được khởi động lần đầu.

Khi một người sử dụng triệu hồi một Servlet, một đối tượng đơn của Servlet được tạo, với mỗi yêu cầu từ người dùng, là kết quả trong một thread mới, mà được thao tác tới doGet hoặc doPost một cách thích hợp. Phương thức init() tạo hoặc tải một vài dữ liệu mà sẽ được sử dụng qua vòng đời của Servlet đó.

Định nghĩa phương thức init() như sau:

```
public void init() throws ServletException {
    // Initialization code...
}
```

Phương thức service()

Phương thức service() là phương thức chính để thực hiện tác vụ thực sự. Nơi chứa servlet (ví dụ như Web Server) gọi phương thức service() để xử lý các yêu cầu từ Client (hoặc trình duyệt) và viết phản hồi đã được định dạng trở lại Client đó.

Mỗi khi Server nhận một yêu cầu cho một Servlet, thì Server tạo một Thread mới và triệu hồi service(). Phương thức service() kiểm tra kiểu yêu cầu HTTP (Kiểu GET, POST, PUT, DELET, .v.v.) và gọi các phương thức doGet, doPost, doPut, doDelete .v.v. tương ứng một cách thích hợp.

Đây là minh họa cho phương thức service():

```
public void service(ServletRequest request,
    ServletResponse response)
    throws ServletException, IOException{
```

```
}
```

Phương thức `service()` được gọi bởi Container và nó triệu hồi các phương thức `doGet`, `doPost`, `doPut`, `doDelete`, .v.v. tương ứng. Vì thế, ta không phải làm gì cả với `service()` nhưng việc ta ghi đè phương thức hoặc `doGet()` hoặc `doPost` tùy thuộc vào kiểu yêu cầu mà ta nhận từ Client.

Hai phương thức `doGet()` và `doPost()` được sử dụng thường xuyên nhất với mỗi `service`. Sau đây là chi tiết về hai phương thức này:

Phương thức `doGet()`

Một yêu cầu GET, là kết quả từ một yêu cầu chuẩn cho URL hoặc từ một HTML form, mà không có PHƯƠNG THỨC nào được xác định và nó nên được xử lý bởi phương thức `doGet()`.

```
public void doGet(HttpServletRequest request,
                  HttpServletResponse response)
    throws ServletException, IOException {
    // Servlet code
}
```

Phương thức `doPost()`

Một yêu cầu POST, là kết quả từ một HTML form mà liệt kê POST như là phương thức, và nên được xử lý bởi phương thức `doPost()`:

```
public void doPost(HttpServletRequest request,
                   HttpServletResponse response)
    throws ServletException, IOException {
    // Servlet code
}
```

Phương thức `destroy()`

Phương thức `destroy()` chỉ được gọi một lần ở giai đoạn cuối trong vòng đời Servlet. Phương thức này giúp servlet của ta một cơ hội để đóng các kết nối tới Database, dừng thread, viết các danh sách cookie hoặc viết tính toán trên đĩa, và thực hiện các hoạt động cleanup khác.

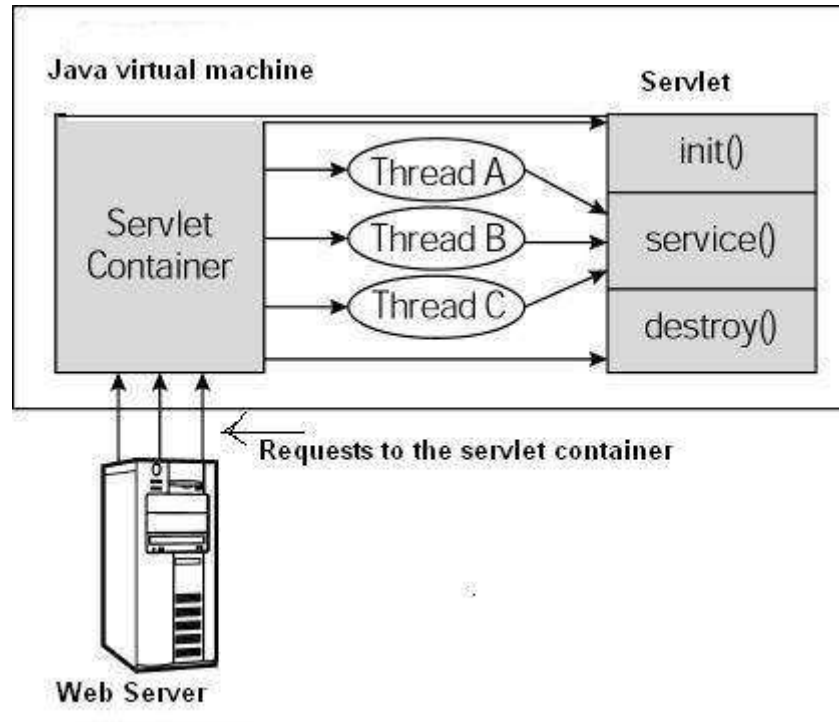
Sau khi phương thức `destroy()` được gọi, đối tượng servlet này được đánh dấu cho Garbage Collector. Phương thức này trông giống như sau:

```
public void destroy() {
    // Finalization code...
}
```

Sơ đồ cấu trúc vòng đời của Servlet

- Sơ đồ sau miêu tả các giai đoạn trong vòng đời của một Servlet đặc trưng:
- Đầu tiên, các HTTP Request tới Server và được đưa tới Container của Servlet.
- Container Servlet tải các servlet trước khi gọi phương thức service().

Sau đó Container Servlet xử lý nhiều yêu cầu bởi việc tạo nhiều thread, mỗi thread thực thi phương thức service() cho một đối tượng servlet đơn.



Hình 3: 4 Sơ đồ cấu trúc vòng đời của Servlet

Ví dụ về Servlet

Servlet là các lớp Java mà phục vụ các yêu cầu HTTP và triển khai `javax.servlet.Servlet` interface. Các lập trình viên ứng dụng web viết các Servlet mà kế thừa `javax.servlet.http.HttpServlet`, một lớp abstract mà triển khai Servlet Interface và được thiết kế riêng biệt để xử lý các yêu cầu HTTP.

Code mẫu cho Hello World

Dưới đây là cấu trúc code mẫu cho một ví dụ servlet để viết Hello World:

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {
    private String message;
```

```

public void init() throws ServletException
{
    // Do required initialization
    message = "Hello World";
}

public void doGet(HttpServletRequest request,
                  HttpServletResponse response)
    throws ServletException, IOException
{
    // Set response content type
    response.setContentType("text/html");
    // Actual logic goes here.
    PrintWriter out = response.getWriter();
    out.println("<h1>" + message + "</h1>");
}

public void destroy()
{
    // do nothing.
}
}

```

3.4. Cấu hình Servlet

Theo mặc định, một ứng dụng servlet được đặt tại path <Tomcat-installation-directory>/webapps/ROOT và class file sẽ ở trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/classes.

Nếu đã có tên class đầy đủ của com.myorg.MyServlet, thì lớp servlet này phải được đặt trong WEB-INF/classes/com/myorg/MyServlet.class.

Bây giờ, sao chép HelloWorld.class vào trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/classes và tạo các entry sau trong web.xml file, được đặt trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/

```

<servlet>
    <servlet-name>HelloWorld</servlet-name>
    <servlet-class>HelloWorld</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>HelloWorld</servlet-name>
    <url-pattern>/HelloWorld</url-pattern>
</servlet-mapping>

```

Các entry trên để được tạo bên trong các thẻ `<web-app>...</web-app>` có sẵn trong web.xml file. Có nhiều entry đa dạng có sẵn trong đó, nhưng đừng lo lắng về điều này.

Điều cần làm bây giờ, khởi động Tomcat server bởi sử dụng `<Tomcat-installation-directory>\bin\startup.bat` (trên windows) hoặc `<Tomcat-installation-directory>/bin/startup.sh` (trên Linux/Solaris .v.v.) và cuối cùng gõ `http://localhost:8080/HelloWorld` trong hộp địa chỉ của trình duyệt. Nếu mọi thứ là ổn, ta sẽ thấy kết quả như sau:



Hình 3: 5 Kết quả chương trình Hello Servlet

3.5. Tham số khởi tạo Servlet

Khi khai báo servlet trong web.xml ta có thể truyền các tham số khởi tạo cho nó. Để truyền tham số khởi tạo cho Servlet

```
<servlet>
  <servlet-name>Servletname</servlet-name>
  <servlet-class>Servletclass</servlet-class>
  <init-param>
    <param-name> paramname1 </param-name>
    <param-value> paramvalue1</param-value>
  </init-param>
  <init-param>
    <param-name> paramname2 </param-name>
    <param-value> paramvalue2</param-value>
  </init-param>
</servlet>
```

Để lấy về tham số khởi tạo ta có thể thông qua hai cách:

1. thông qua đối tượng của lớp ServletConfig của phương thức `init()`;

2. thông qua phương thức `config.getInitParameter("paramname1");`
`this.getServletConfig().getInitParameter("paramname2");`

Ví dụ: `web.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" id="WebApp_ID"
version="3.1">
  <display-name>TruyenThamBien</display-name>
  <servlet>
    <servlet-name>ViduTruyenthambien</servlet-name>
    <servlet-class>abc.ViduTruyenthambien</servlet-class>
    <init-param>
      <param-name>bien1</param-name>
      <param-value>homnay0912@gmail.com</param-value>
    </init-param>
    <init-param>
      <param-name>bien2</param-name>
      <param-value>tonguyencntt@yahoo.com</param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>ViduTruyenthambien</servlet-name>
    <url-pattern>/vidutruyenthambien</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.jsp</welcome-file>
    <welcome-file>default.htm</welcome-file>
  </welcome-file-list>
</web-app>
```

Chương trình Servlet thực hiện đọc các tham biến:

```
import java.io.IOException;
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletOutputStream;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/ViduTruyenthambien")
public class ViduTruyenthambien extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private String bien1;
    public ViduTruyenthambien() {
        super();
        // TODO Auto-generated constructor stub
    }
}
```

```

    }
    public void init(ServletConfig config) throws ServletException {
        // TODO Auto-generated method stub
        super.init(config);
        bien1=config.getInitParameter("bien1");
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        response.getWriter().append("Served at:
").append(request.getContextPath());
        String bien2=this.getServletConfig().getInitParameter("bien2");
        //String bien3 =
this.getServletConfig().getInitParameter("bien2");
        ServletOutputStream out=response.getOutputStream();
        out.println("<html>");
        out.println("<head><title>Init Param</title></head>");
        out.println("<body>");
        out.println("<h3>Init Param</h3>");
        out.println("<p>emailSupport1 = " + this.bien1 + "</p>");
        out.println("<p>emailSupport2 = " + bien2 + "</p>");
        out.println("</body>");
        out.println("<html>");
    }
    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
        doGet(request, response);
    }
}

```

3.6. Mapping Servlet

-web.xml

Web.xml được định nghĩa ánh xạ giữa đường dẫn URL và servlet xử lý các yêu cầu với các đường dẫn đó. File web.xml cung cấp thông tin triển khai và cấu hình cho các thành phần Web bao gồm một ứng dụng Web. Các file servlet được mô tả trong web.xml. Nó đại diện cho cốt lõi của ứng dụng java. File web.xml nằm trong thư mục WEB-INF của ứng dụng web.

-Các phần tử trong Web.xml

Mô tả các phần tử được sử dụng trong file web.xml:

1. **<web-app>**: Phần tử này đại diện cho toàn bộ ứng dụng của file web.xml
2. **<servlet>** : Đây là phần tử con của và đại diện cho servlet
3. **<servlet-name>** : Đây là phần tử con của servlet và được sử dụng để đại diện cho tên của servlet
4. **<servlet-class>** : Đây là phần tử con của servlet và được sử dụng để đại diện cho lớp servlet.
5. **<servlet-mapping>** : Đây là phần tử con của và được sử dụng để ánh xạ servlet
6. **<url-pattern>** : Đây là phần tử con của servlet-mapping và được sử dụng cho phía Máy khách để gọi servlet, hay nói cách khác là url tương ứng cho servlet.

Giải thích thêm thì thẻ <welcome-file-list> dùng để liệt kê các file khi một ứng dụng web chạy lên, những file nằm trong danh sách thẻ <welcome-file-list> được ưu tiên gọi nếu có.

3.7. Sử dụng Annotations để khai báo servlets, Filter và listeners

Với Servlet phiên bản 3.0 trở lên ta có thể cấu hình Servlet sử dụng Annotation, trong ví dụ này chúng ta đang sử dụng Servlet phiên bản 3.x vì vậy chúng ta có thể sử dụng Annotation để cấu hình.

Mục đích tạo ra các chú thích annotation này là để các nhà phát triển dễ dàng hơn trong quá trình làm việc với servlet như mapping các servlet hay các filter.

Cùng nhìn 2 ví dụ bên dưới:

Trong servlets 2.5, để khai báo một servlet có một tham số init, ta cần thêm phần này vào web.xml:

```
<servlet>
    <servlet-name>myServlet</servlet-name>
    <servlet-class>my.server.side.stuff.MyAwesomeServlet</servlet-
class>
    <init-param>
        <param-name>configFile</param-name>
        <param-value>config.xml</param-value>
    </init-param>
</servlet>
<servlet-mapping>
    <servlet-name>myServlet</servlet-name>
    <url-pattern>/path/to/my/servlet</url-pattern>
</servlet-mapping>
```

Trong servlet 3.0, web.xml là tùy chọn và có thể sử dụng các chú thích annotation thay vì XML. Ví dụ tương tự:

```
@WebServlet(name="myServlet",
    urlPatterns={"/path/to/my/servlet"},
    initParams={@InitParam(name="configFile", value="config.xml")})
public class MyAwesomeServlet extends HttpServlet { ... }
```

Đối với các filter, cần thêm phần này vào web.xml trong servlets 2.5:

```

<filter>
    <filter-name>myFilter</filter-name>
    <filter-class>my.server.side.stuff.MyAwesomeServlet</filter-cl
ass>
</filter>
<filter-mapping>
    <filter-name>myFilter</filter-name>
    <url-pattern>/path/to/my/filter</url-pattern>
</filter-mapping>

```

Đối với web servlet 3.0 được đơn giản hóa như sau:

```

@WebServlet(name="myFilter", urlPatterns={"/path/to/my/filter"})
)
public class MyAwesomeFilter implements Filter { ... }

```

Đối với listener (trong trường hợp này là ServletContextListener), trong servlets 2.5:

```

<listener>
<listener-class>my.server.side.stuff.MyAwesomeListener</listener-c
lass>
</listener>

```

Với servlet 3.0 được đặc tả như sau:

```

@WebServletContextListener
public class MyAwesomeListener implements ServletContextListener {
... }

```

3.8. Form Data trong Servlet

Khi gặp phải nhiều tình huống mà cần truyền một số thông tin từ trình duyệt Web Server và sau đó tới chương trình backend. Trình duyệt sử dụng hai phương thức để truyền thông tin này tới Web Server. Đó là phương thức GET và phương thức POST.

3.8.1. Phương thức doGET

Phương thức GET gửi thông tin người dùng đã được mã hóa phụ thêm tới yêu cầu trang. Trang này và thông tin đã được mã hóa được phân biệt bởi ký tự ? như sau:

```
http://www.test.com/hello?key1=value1&key2=value2
```

Phương thức GET là phương thức mặc định để truyền thông tin từ trình duyệt tới Web Server và nó tạo ra một chuỗi dài mà xuất hiện trong Location:box của trình duyệt. Không bao giờ sử dụng phương thức GET nếu truyền tới Server thông tin

password hoặc thông tin nhạy cảm khác. Phương thức GET có kích cỡ giới hạn: chỉ có 1024 ký tự trong một chuỗi yêu cầu.

Thông tin này được truyền bởi sử dụng QUERY_STRING Header và sẽ có thể truy cập thông qua biến môi trường QUERY_STRING và Servlet xử lý kiểu yêu cầu này bởi sử dụng phương thức doGet().

Phương thức POST

Một phương thức đáng tin cậy hơn để truyền thông tin tới chương trình backend là phương thức POST. Nó gói thông tin theo cách giống như phương thức GET, nhưng thay vì gửi nó như là một chuỗi text sau một ký tự ? trong URL, nó gửi thông tin như là một thông báo riêng biệt. Thông báo này tới chương trình backend trong form của input chuẩn mà ta có thể parse và sử dụng cho tiến trình xử lý. Servlet xử lý kiểu yêu cầu này bởi sử dụng phương thức doPost().

Đọc Form Data bởi sử dụng Servlet

Servlet xử lý Form Data bằng việc parse một cách tự động bởi sử dụng các phương thức sau tùy thuộc vào từng tình huống cụ thể:

- **getParameter():** Ta gọi phương thức request.getParameter() để nhận giá trị của tham số Form.
- **getParameterValues():** Gọi phương thức này nếu parameter xuất hiện nhiều hơn một lần và trả về nhiều giá trị, ví dụ như checkbox.
- **getParameterNames():** Gọi phương thức này nếu ta muốn một danh sách đầy đủ của tất cả các parameter trong yêu cầu hiện tại.

Ví dụ phương thức GET sử dụng URL

Sau đây là một URL đơn giản mà sẽ truyền hai giá trị tới chương trình HelloForm bởi sử dụng phương thức GET:

- **http://localhost:8080/HelloForm?first_name=ZARA&last_name=ALI**

Chương trình servlet sau với tên HelloForm.java để xử lý đầu vào đã được cung cấp bởi trình duyệt web. Chúng ta đang sử dụng phương thức getParameter() để làm cho việc truy cập thông tin đã truyền trở nên đơn giản hơn:

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloForm extends HttpServlet {
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
```

```

{
    // Set response content type
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String title = "Using GET Method to Read Form Data";
    String docType =
        "<!doctype html public \"-//w3c//dtd html 4.0 \" +
        \"transitional//en\">\n";
    out.println(docType +
        "<html>\n" +
        "<head><title>" + title + "</title></head>\n" +
        "<body bgcolor=\"#f0f0f0\">\n" +
        "<h1 align=\"center\">" + title + "</h1>\n" +
        "<ul>\n" +
        "  <li><b>First Name</b>: "
        + request.getParameter("first_name") + "\n" +
        "  <li><b>Last Name</b>: "
        + request.getParameter("last_name") + "\n" +
        "</ul>\n" +
        "</body></html>");
}
}

```

Khi môi trường đã được cài đặt thích hợp, biên dịch HelloForm.java như sau:

```
$ javac HelloForm.java
```

Nếu mọi thứ là ổn, nó sẽ tạo HelloForm.class file. Sau đó ta phải sao chép class file này vào trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/classes và tạo các entry sau trong web.xml file được đặt trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/

```

<servlet>
    <servlet-name>HelloForm</servlet-name>
    <servlet-class>HelloForm</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>HelloForm</servlet-name>
    <url-pattern>/HelloForm</url-pattern>

```

```
</servlet-mapping>
```

`http://localhost:8080/HelloForm?first_name=ZARA&last_name=ALI` trong

Location:box của trình duyệt và đảm bảo rằng ta đã bắt đầu Tomcat Server, trước khi kích hoạt lệnh trên trong trình duyệt. Nó sẽ cho kết quả sau:

Using GET Method to Read Form Data

First Name: ZARA

Last Name: ALI

3.8.2. Phương thức GET sử dụng FORM

Ví dụ đơn giản sau sẽ truyền hai giá trị bởi sử dụng HTML FORM và nút submit. Chúng ta cũng đang sử dụng Servlet HelloForm để xử lý input này.

```
<html>
<body>
<form action="HelloForm" method="GET">
First Name: <input type="text" name="first_name">
<br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>
</body>
</html>
```

First Name: Last Name:

Bottom of Form

Khi First Name và Last Name và sau đó vào nút submit để xem kết quả trên thiết bị nội bộ, nơi Tomcat đang chạy. Dựa trên input đã cung cấp, nó sẽ tạo ra cùng kết quả như đã nói trong ví dụ trên.

Truyền Checkbox Data tới chương trình Servlet

Các checkbox được sử dụng khi có nhiều hơn một tùy chọn để được lựa chọn.

HTML code sau, với tên CheckBox.htm, cho một form với hai checkbox.

```

<html>
<body>
<form action="CheckBox" method="POST" target="_blank">
<input type="checkbox" name="maths" checked="checked" /> Maths
<input type="checkbox" name="physics" /> Physics
<input type="checkbox" name="chemistry" checked="checked" />
Chemistry

<input type="submit" value="Select Subject" />
</form>
</body>
</html>

```

Kết quả là form sau:

☒ Maths ☐ Physics ☒ Chemistry

Chương trình Servlet sau, với tên CheckBox.java, để xử lý input được cung cấp bởi trình duyệt web cho nút checkbox.

```

// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
// Extend HttpServlet class
public class CheckBox extends HttpServlet {
    // Method to handle GET method request.
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // Set response content type
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        String title = "Reading Checkbox Data";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";
    }
}

```

```

        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n" +
            "<body bgcolor=\"#f0f0f0\">\n" +
            "<h1 align=\"center\">" + title + "</h1>\n" +
            "<ul>\n" +
            "  <li><b>Maths Flag : </b>: "
            + request.getParameter("maths") + "\n" +
            "  <li><b>Physics Flag: </b>: "
            + request.getParameter("physics") + "\n" +
            "  <li><b>Chemistry Flag: </b>: "
            + request.getParameter("chemistry") + "\n" +
            "</ul>\n" +
            "</body></html>");
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}

```

Với ví dụ trên, nó hiển thị kết quả:

Reading Checkbox Data

Maths Flag : : on

Physics Flag: : null

Chemistry Flag: : on

Đọc tất cả Form Parameter

Ví dụ chung sau đây sử dụng phương thức `getParameterNames()` của `HttpServletRequest` để đọc tất cả Form Parameter có sẵn. Phương thức này trả về một Enumeration (bản liệt kê) mà chứa các tên Parameter trong thứ tự chưa được sắp xếp.

Khi chúng ta có một Enumeration, lặp Enumeration này theo phương thức chuẩn: sử dụng phương thức `hasMoreElements()` để xác định khi nào cần dừng và sử dụng phương thức `nextElement()` để nhận tên của mỗi parameter.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class ReadParams extends HttpServlet {
    // Method to handle GET method request.
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // Set response content type
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        String title = "Reading All Form Parameters";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";
        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n" +
            "<body bgcolor=\"#f0f0f0\">\n" +
            "<h1 align=\"center\">" + title + "</h1>\n" +
            "<table width=\"100%\" border=\"1\" align=\"center\">\n" +
            "<tr bgcolor=\"#949494\">\n" +
            "<th>Param Name</th><th>Param Value(s)</th>\n" +
            "</tr>\n");
        Enumeration paramNames = request.getParameterNames();
        while(paramNames.hasMoreElements()) {
            String paramName = (String)paramNames.nextElement();
            out.print("<tr><td>" + paramName + "</td>\n<td>");
```

```

        String[] paramValues =
            request.getParameterValues(paramName);
        // Read single valued data
        if (paramValues.length == 1) {
            String paramValue = paramValues[0];
            if (paramValue.length() == 0)
                out.println("<i>No Value</i>");
            else
                out.println(paramValue);
        } else {
            // Read multiple valued data
            out.println("<ul>");
            for(int i=0; i < paramValues.length; i++) {
                out.println("<li>" + paramValues[i]);
            }
            out.println("</ul>");
        }
    }
    out.println("</tr>\n</table>\n</body></html>");
}

// Method to handle POST method request.
public void doPost(HttpServletRequest request, HttpServletResponse re
sponse)
    throws ServletException, IOException {
    doGet(request, response);
}
}

```

Servlet trên với form sau:

```

<html>
<body>
<form action="ReadParams" method="POST" target="_blank">
<input type="checkbox" name="maths" checked="checked" /> Maths
<input type="checkbox" name="physics" /> Physics
<input type="checkbox" name="chemistry" checked="checked" /> Chem
<input type="submit" value="Select Subject" />
</form>

```

```
</body>
</html>
```

Bây giờ gọi Servlet bởi sử dụng form trên sẽ cho kết quả sau:

Reading All Form Parameters

Param Name	Param Value(s)
maths	on
chemistry	on

Ta cũng có thể thử Servlet trên để đọc bất kỳ dữ liệu của Form khác mà có các đối tượng khác như Text box, Radio button, Dropdown box .v.v.

3.8.3. Client Request trong Servlet

Khi một trình duyệt yêu cầu một trang web, nó gửi nhiều thông tin tới Web Server, mà không thể được đọc một cách trực tiếp bởi vì thông tin này chuyển động như là một phần của Header trong HTTP Request. Ta có thể kiểm tra Giao thức HTTP để tìm thêm thông tin về điều này.

Bảng dưới liệt kê các thông tin Header quan trọng mà tới từ trình duyệt và ta sẽ sử dụng rất thường xuyên trong lập trình Web.

Header	Miêu tả
Accept	Trường này xác định loại MIME mà trình duyệt hoặc Client khác có thể xử lý. Các giá trị image/png hoặc image/jpeg là hai khả năng phổ biến nhất
Accept-Charset	Trường này xác định bộ ký tự mà trình duyệt có thể sử dụng để hiển thị thông tin. Ví dụ: ISO-8859-1
Accept-Encoding	Trường này xác định kiểu mã hóa mà trình duyệt biết để xử lý. Các giá trị của gzip hoặc compress là hai khả năng phổ biến nhất
Accept-Language	Trường này xác định ngôn ngữ ưa thích của Client trong trường hợp Servlet có thể tạo các kết quả trong nhiều ngôn ngữ. Ví dụ en, en-us, ru, .v.v.

Authorization	Trường này được sử dụng bởi Client để nhận diện chính nó khi truy cập vào các trang web được bảo vệ bởi password
Connection	Trường này chỉ dẫn có hay không Client có thể xử lý các kết nối HTTP dạng Persistent. Các kết nối Persistent cho phép Client hoặc trình duyệt thu nhận nhiều file với một yêu cầu đơn. Một giá trị của Keep-Alive nghĩa là các kết nối Persistent này nên được sử dụng
Content-Length	Trường này chỉ thích hợp với các yêu cầu POST và cung cấp kích cỡ của dữ liệu POST bằng giá trị byte
Cookie	Trường này trả về các cookie tới các Server mà trước đó đã gửi chúng tới trình duyệt
Host	Trường này xác định host và cổng (port) khi được cung cấp trong URL ban đầu
If-Modified-Since	Trường này chỉ rằng Client muốn trang này chỉ khi nó đã được thay đổi sau ngày đã xác định. Server gửi một code là 304, nghĩa là Not Modified Header nếu không có kết quả nào mới hơn là có sẵn
If-Unmodified-Since	Trường này là ngược với trường If-Modified-Since. Nó xác định hoạt động là thành công chỉ khi tài liệu là cũ hơn ngày đã xác định
Referer	Trường này chỉ URL của trang web đang tham chiếu đó. Ví dụ, nếu ta tại trang Webpage 1 và click vào một link tới trang Webpage 2, thì URL của Webpage 1 được bao trong trường Referer Header khi trình duyệt yêu cầu trang Webpage 2
User-Agent	Trường này nhận diện trình duyệt hoặc Client đang tạo yêu cầu và có thể được sử dụng để trả về nội dung khác nhau cho các loại trình duyệt khác nhau

Bảng 3: 1 Các thông tin Header từ trình duyệt

HttpServletRequest

Các phương thức để đọc HTTP Header

Bảng dưới liệt kê các phương thức có thể được sử dụng để đọc HTTP Header trong chương trình Servlet. Các phương thức này có sẵn với đối tượng HttpServletRequest.

STT	Phương thức & Miêu tả
1	Cookie[] getCookies() Trả về một mảng chứa tất cả đối tượng Cookie mà Client gửi với yêu cầu này
2	Enumeration getAttributeNames() Trả về một Enumeration chứa các tên của thuộc tính có sẵn cho yêu cầu này
3	Enumeration getHeaderNames() Trả về một Enumeration của tất cả tên Header mà yêu cầu này chứa
4	Enumeration getParameterNames() Trả về một Enumeration của các đối tượng String chứa tên của Parameter được chứa trong yêu cầu này
5	HttpSession getSession() Trả về session hiện tại, mà liên kết với yêu cầu này, hoặc yêu cầu đó không có một session, nó sẽ tạo ra một session
6	HttpSession getSession(boolean create) Trả về HttpSession hiện tại, mà liên kết với yêu cầu này, hoặc nếu không có session hiện tại nào và create là true, nó trả về một session mới
7	Locale getLocale() Trả về Locale mà Client sẽ chấp nhận nội dung trong đó, dựa trên trường Accept-Language Header
8	Object getAttribute(String name) Trả về giá trị của thuộc tính name như là một Object, hoặc trả về null nếu không có thuộc tính nào trong name đã cung cấp tồn tại
9	ServletInputStream getInputStream() Thu nhận thân yêu cầu dạng dữ liệu nhị phân bởi sử dụng một ServletInputStream

10	String getAuthType() Trả về tên của Authentication Scheme được sử dụng để bảo vệ Servlet, ví dụ: "BASIC" hoặc "SSL", hoặc trả về null nếu JSP không được bảo vệ
11	String getCharacterEncoding() Trả về tên của Mã hóa ký tự được sử dụng trong thân yêu cầu này
12	String getContentType() Trả về kiểu MIME của thân yêu cầu, hoặc trả về null nếu không biết kiểu này
13	String getContextPath() Trả về phần URI yêu cầu mà chỉ dẫn context của yêu cầu đó
14	String getHeader(String name) Trả về giá trị của trường Header đã xác định dưới dạng một String
15	String getMethod() Trả về tên của phương thức HTTP mà yêu cầu này được tạo, ví dụ: GET, POST, hoặc PUT
16	String getParameter(String name) Trả về giá trị của Parameter dưới dạng một String, hoặc trả về null nếu Parameter này không tồn tại
17	String getPathInfo() Trả về bất kỳ thông tin path bổ sung, mà liên kết với URI mà Client đã gửi khi nó tạo yêu cầu này
18	String getProtocol() Trả về tên và phiên bản của giao thức
19	String getQueryString() Trả về chuỗi truy vấn, được chứa trong URL sau path đó

20	String getRemoteAddr() Trả về địa chỉ Internet Protocol (IP) của một Client mà gửi yêu cầu này
21	String getRemoteHost() Trả về tên đầy đủ của Client mà gửi yêu cầu này
22	String getRemoteUser() Trả về login của người sử dụng tạo yêu cầu này, nếu người dùng đã được xác nhận, hoặc trả về null nếu người dùng chưa được xác nhận
3	2 String getRequestedURI() Trả về phần URL của yêu cầu này từ tên giao thức dưới dạng một chuỗi truy vấn trong dòng đầu tiên của HTTP Request
4	2 String getRequestedSessionId() Trả về ID của session đã được xác định bởi Client
5	2 String getServletPath() Trả về phần URL của yêu cầu này mà gọi JSP
6	2 String[] getParameterValues(String name) Trả về một mảng đối tượng String chứa tất cả các giá trị mà các parameter đã cung cấp có, hoặc trả về null nếu parameter này không tồn tại
7	2 boolean isSecure() Trả về một Boolean chỉ rằng có hay không yêu cầu này được tạo bởi sử dụng một đường an toàn, như HTTPS
8	2 int getContentLength() Trả về độ dài, bằng giá trị byte, của thân yêu cầu và làm nó có sẵn bởi Input Stream, hoặc trả về -1 nếu độ dài là không được biết
9	2 int getIntHeader(String name) Trả về giá trị của Request header đã cho dưới dạng một int

0	3	int getServerPort() Trả về số hiệu cổng mà trên đó yêu cầu được nhận
---	---	--

Bảng 3: 2 Các phương thức của HttpServletRequest

Ví dụ về HTTP Header Request

Ví dụ sau sử dụng phương thức `getHeaderName()` của đối tượng `HttpServletRequest` để đọc thông tin HTTP header. Phương thức này trả về một Enumeration chứa thông tin header mà liên kết với HTTP request hiện tại.

Khi chúng ta có một Enumeration, chúng ta có thể lặp Enumeration này theo phương thức chuẩn: sử dụng phương thức `hasMoreElements()` để xác định khi nào dừng và sử dụng phương thức `nextElement()` để nhận tên mỗi parameter.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class DisplayHeader extends HttpServlet {
    // Method to handle GET method request.
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // Set response content type
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String title = "HTTP Header Request Example";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";
        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n" +
            "<body bgcolor=\"#f0f0f0\">\n" +
            "<h1 align=\"center\">" + title + "</h1>\n" +
```

```

        "<table width=\"100%\" border=\"1\" align=\"center\">\n" +
        "<tr bgcolor=\"#949494\">\n" +
        "<th>Header Name</th><th>Header Value(s)</th>\n"+
        "</tr>\n");

        Enumeration headerNames = request.getHeaderNames();

        while(headerNames.hasMoreElements()) {
            String paramName = (String)headerNames.nextElement();
            out.print("<tr><td>" + paramName + "</td>\n");
            String paramValue = request.getHeader(paramName);
            out.println("<td> " + paramValue + "</td></tr>\n");
        }
        out.println("</table>\n</body></html>");
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {    doGet(request, resp
onse);
    }
}

```

Bây giờ gọi Servlet trên sẽ cho kết quả sau:

HTTP Header Request Example

Header Name	Header Value(s)
accept	*/*
accept-language	en-us
user-agent	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0;

	InfoPath.2; MS-RTC LM 8)
accept-encoding	gzip, deflate
host	localhost:8080
connection	Keep-Alive
cache-control	no-cache

3.8.4. Server Response trong Servlet

Như đã bàn luận trong chương trước, khi một Web Server phản hồi một HTTP Request tới trình duyệt, phản hồi đặc trưng bao gồm một dòng Status, một số trường Header, một dòng trống, và tài liệu. Một phản hồi đặc trưng giống như sau:

```
HTTP/1.1 200 OK
Content-Type: text/html
Header2: ...
...
HeaderN: ...
(Blank Line)
<!doctype ...>
<html>
<head>...</head>
<body>
...
</body>
</html>
```

Dòng Status gồm HTTP version (trong ví dụ là HTTP/1.1), một Status code (trong ví dụ là 200), và một thông báo rất ngắn tương ứng với Status code (trong ví dụ là OK).

Dưới đây là các trường Header hữu ích nhất trong phản hồi HTTP 1.1 mà trở lại trình duyệt từ Web Server và ta sẽ thường xuyên sử dụng chúng trong lập trình web:

Header	Miêu tả
--------	---------

Allow	Trường này xác định các phương thức yêu cầu (GET, POST, .v.v.) mà Server hỗ trợ
Cache-Control	Trường này xác định các hoàn cảnh mà trong đó tài liệu phản hồi có thể được cache một cách an toàn. Nó có thể có các giá trị public , private hoặc no-cache , .v.v.. Trong đó, public nghĩa là tài liệu có thể được cache, private nghĩa là tài liệu dành cho một người sử dụng và chỉ có thể được lưu giữ riêng (không được chia sẻ) và no-cache nghĩa là tài liệu không bao giờ được cache
Connection	Trường này chỉ dẫn trình duyệt có sử dụng persistent trong các kết nối HTTP hoặc không. Một giá trị close chỉ rằng trình duyệt không sử dụng các kết nối HTTP dạng persistent và keep-alive nghĩa là sử dụng các kết nối persistent
Content-Disposition	Trường này cho ta yêu cầu rằng trình duyệt hỏi người sử dụng để lưu phản hồi lên đĩa trong một file với tên đã cho
Content-Encoding	Trường này xác định cách mà trang được mã hóa trong khi truyền tải
Content-Language	Trường này xác định ngôn ngữ mà tài liệu được viết. Ví dụ en, en-us, ru, ...
Content-Length	Trường này xác định số byte trong phản hồi. Thông tin này chỉ cần thiết nếu trình duyệt đang sử dụng một kết nối HTTP dạng persistent (giá trị keep-alive)
Content-Type	Trường này kiểu MIME (Multipurpose Internet Mail Extension) của tài liệu phản hồi
Expires	Trường này xác định thời gian tại đó nội dung nên được xem là hết hạn và không bao giờ được cache nữa
Last-Modified	Trường này chỉ dẫn khi tài liệu được thay đổi lần cuối. Sau đó Client có thể cache tài liệu này và cung cấp một date bởi một trường yêu cầu If-Modified-Since trong các yêu cầu sau đó
Location	Trường này nên được bao với tất cả phản hồi mà có Status code là 300. Nó thông báo cho trình duyệt về địa chỉ tài liệu. Trình duyệt tự động kết nối lại tới vị trí này và thu nhận tài liệu mới

Refresh	Trường này xác định khi nào trình duyệt nên đề nghị một trang đã được update. Ta có thể xác định thời gian với đơn vị là số giây sau đó một trang sẽ được refresh
Retry-After	Trường này có thể được sử dụng khi kết hợp với phản hồi 503 (Service Unavailable) để nói cho Client khi nào thì nó có thể lặp lại yêu cầu
Set-Cookie	Trường này xác định một cookie mà liên kết với trang

Bảng 3: 3 Các trường Header
Các phương thức để thiết lập HTTP Response Header

Bảng dưới liệt kê các phương thức có thể được sử dụng để thiết lập HTTP Response Header trong chương trình Servlet. Các phương thức này có sẵn với đối tượng HttpServletResponse.

STT	Phương thức và Miêu tả
1	String encodeRedirectURL(String url) Mã hóa URL đã cho để sử dụng trong phương thức sendRedirect, hoặc nếu mã hóa này không được đòi hỏi, thì trả về URL như cũ
2	String encodeURL(String url) Mã hóa URL đã cho bằng việc thêm ID của session trong nó, hoặc nếu mã hóa này không được đòi hỏi, thì trả về URL như cũ
3	boolean containsHeader(String name) Trả về một Boolean chỉ rằng có hay không header phản hồi với tên đã cho đã được thiết lập
4	boolean isCommitted() Trả về một Boolean chỉ rằng nếu phản hồi đã được commit
5	void addCookie(Cookie cookie) Thêm cookie đã cho tới phản hồi
6	void addDateHeader(String name, long date) Thêm một header phản hồi với tên và giá trị date đã cho
7	void addHeader(String name, String value) Thêm một header phản hồi với tên và giá trị đã cho

8	void addIntHeader(String name, int value) Thêm một header phản hồi với tên và giá trị integer đã cho
9	void flushBuffer() Force bất kỳ nội dung nào trong Buffer để được ghi tới Client đó
10	void reset() Xóa bất dữ liệu nào tồn tại trong buffer cũng như Status code và các trường header
11	void resetBuffer() Xóa nội dung của buffer nằm dưới trong phản hồi mà không xóa các trường header và status code
12	void sendError(int sc) Gửi một phản hồi lỗi tới Client bởi sử dụng Status code đã cho và xóa buffer
13	void sendError(int sc, String msg) Gửi một phản hồi lỗi tới Client bởi sử dụng Status code đã cho
14	void sendRedirect(String location) Gửi một phản hồi Redirect tạm thời tới Client bởi sử dụng địc chỉ URL để Redirect đã cho
15	void setBufferSize(int size) Thiết lập kích cỡ buffer cho thân phản hồi
16	void setCharacterEncoding(String charset) Thiết lập mã hóa ký tự (MIME Charset) của phản hồi đang được gửi tới Client, ví dụ, thiết lập là UTF-8
17	void setContentLength(int len) Thiết lập độ dài nội dung của thân phản hồi trong HTTP Servlet, phương thức này thiết lập trường header là HTTP Content-Length
18	void setContentType(String type) Thiết lập kiểu nội dung của phản hồi đang được gửi tới Client, nếu phản hồi vẫn chưa được commit
19	void setDateHeader(String name, long date) Thiết lập một header phản hồi với tên và giá trị date đã cho

20	void setHeader(String name, String value) Thiết lập một header phản hồi với tên và giá trị đã cho
21	void setIntHeader(String name, int value) Thiết lập một header phản hồi với tên và giá trị integer đã cho
22	void setLocale(Locale loc) Thiết lập Locale của phản hồi, nếu phản hồi vẫn chưa được commit
23	void setStatus(int sc) Thiết lập Status code cho phản hồi này

Bảng 3: 4 Các phương thức thiết lập Response Header trong Servlet
Ví dụ về HTTP Header Response

Ta đã thấy cách phương thức `setContentType()` làm việc trong ví dụ trước và ví dụ sau cũng sử dụng phương thức này, ngoài ra chúng ta sẽ sử dụng phương thức `setIntHeader()` để thiết lập trường Refresh Header:

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class Refresh extends HttpServlet {

    // Method to handle GET method request.
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // Set refresh, autoload time as 5 seconds
        response.setIntHeader("Refresh", 5);

        // Set response content type
        response.setContentType("text/html");

        // Get current time
```

```

        Calendar calendar = new GregorianCalendar();
        String am_pm;
        int hour = calendar.get(Calendar.HOUR);
        int minute = calendar.get(Calendar.MINUTE);
        int second = calendar.get(Calendar.SECOND);
        if(calendar.get(Calendar.AM_PM) == 0)
            am_pm = "AM";
        else
            am_pm = "PM";
        String CT = hour+":"+ minute +":"+ second + " " + am_pm;
        PrintWriter out = response.getWriter();
        String title = "Auto Refresh Header Setting";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";
        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n"+
            "<body bgcolor=\"#f0f0f0\">\n" +
            "<h1 align=\"center\">" + title + "</h1>\n" +
            "<p>Current Time is: " + CT + "</p>\n");
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}

```

Bây giờ gọi Servlet trên sẽ hiển thị System Time hiện tại sau mỗi 5s như sau. Ta chạy và đợi để xem kết quả:

Auto Refresh Header Setting

Current Time is: 9:44:50 PM

3.8.5. Status Code trong Servlet

Định dạng của các thông báo HTTP request và HTTP response là tương tự nhau và sẽ có cấu trúc sau:

Một dòng status + CRLF (Carriage Return + Line Feed ví dụ: New Line)

0 hoặc nhiều dòng header + CRLF

Một dòng trống, ví dụ: một CRLF

Một thân thông báo tùy ý như file, dữ liệu truy vấn (Query Data) hoặc kết quả truy vấn (Query output).

Ví dụ, một header phản hồi từ Server trông giống như sau:

```
HTTP/1.1 200 OK
Content-Type: text/html
Header2: ...
...
HeaderN: ...
(Blank Line)
<!doctype ...>
<html>
<head>...</head>
<body>
...
</body>
</html>
```

Dòng Status gồm phiên bản HTTP (trong ví dụ là HTTP/1.1), một status code (trong ví dụ là 200), và một thông báo rất ngắn tương ứng với Status code đó (trong ví dụ là OK).

Bảng dưới liệt kê các HTTP Status code và các thông báo tương ứng mà có thể được trả về từ Web Server:

Code	Thông báo	Miêu tả
100	Continue	Chỉ một phần của yêu cầu đã được nhận bởi Server, nhưng chỉ cần nó không bị loại bỏ, Client nên tiếp tục với yêu cầu này
101	Switching Protocols	Server chuyển đổi giữa các giao thức

200	OK	Yêu cầu là OK
201	Created	Yêu cầu là hoàn thành, và một nguồn mới được tạo
202	Accepted	Yêu cầu được chấp nhận để xử lý, nhưng việc xử lý chưa hoàn thành
203	Non-authoritative Information	
204	No Content	
205	Reset Content	
206	Partial Content	
300	Multiple Choices	Một danh sách các link. Người dùng có thể chọn một link và tới trang đó. Tối đa là 5 địa chỉ
301	Moved Permanently	Trang được yêu cầu đã chuyển tới một URL mới
302	Found	Trang được yêu cầu đã chuyển tới một URL mới
303	See Other	Trang được yêu cầu có thể được tìm với một URL khác
304	Not Modified	
305	Use Proxy	
306	<i>Unused</i>	Code này được sử dụng trong một phiên bản trước. Nó không còn được sử dụng nữa, nhưng vẫn được lưu trữ
307	Temporary Redirect	Trang được yêu cầu đã tạm thời chuyển tới một URL mới

400	Bad Request	Server không hiểu yêu cầu
401	Unauthorized	Trang được yêu cầu cần thiết một username và một password
402	Payment Required	<i>Ta không thể sử dụng code này nữa</i>
403	Forbidden	Sự truy cập là bị cấm với trang được yêu cầu
404	Not Found	Server không tìm thấy trang được yêu cầu
405	Method Not Allowed	Phương thức đã được xác định trong yêu cầu là không được cho phép
406	Not Acceptable	Server chỉ có thể tạo một phản hồi mà không được chấp nhận bởi Client
407	Proxy Authentication Required	Ta phải xác nhận với một Proxy server trước khi yêu cầu này có thể được phục vụ
408	Request Timeout	Yêu cầu tồn thời gian quá lâu so với thời gian server đợi
409	Conflict	Yêu cầu này không thể hoàn thành bởi vì có một conflict
410	Gone	Trang được yêu cầu không có sẵn nữa
411	Length Required	"Content-Length" không được định nghĩa. Server sẽ không chấp nhận yêu cầu mà không có nó
412	Precondition Failed	Điều kiện tiên quyết (precondition) đã cung cấp trong yêu cầu được tính toán là false bởi Server
413	Request Entity Too Large	Server sẽ không chấp nhận yêu cầu này, bởi vì đối tượng yêu cầu là quá lớn

414	Request-url Too Long	Server sẽ không chấp nhận yêu cầu này, bởi vì URL là quá dài. Xảy ra khi ta biến đổi một yêu cầu POST thành một yêu cầu GET với một thông tin truy vấn quá dài
415	Unsupported Media Type	Server sẽ không chấp nhận yêu cầu, bởi vì MIME type là không được hỗ trợ
417	Expectation Failed	
500	Internal Server Error	Yêu cầu chưa được hoàn thành. Server gặp một trường hợp không mong muốn
501	Not Implemented	Yêu cầu chưa được hoàn thành. Server không hỗ trợ tính năng được yêu cầu
502	Bad Gateway	Yêu cầu chưa được hoàn thành. Server này nhận một phản hồi không hợp lệ từ Server tuyến trên
503	Service Unavailable	Yêu cầu chưa được hoàn thành. Server tạm thời đang quá tải hoặc hỏng
504	Gateway Timeout	Gateway là hết hạn
505	HTTP Version Not Supported	Server không hỗ trợ phiên bản "http protocol"

Bảng 3: 5 Các HTTP Status code và thông báo tương ứng
Phương thức để thiết lập HTTP Status Code

Dưới đây liệt kê các phương thức có thể được sử dụng để thiết lập HTTP Status code trong chương trình Servlet. Các phương thức này có sẵn với đối tượng HttpServletResponse.

STT	Phương thức & Miêu tả
1	public void setStatus (int statusCode) Phương thức này thiết lập một Status code tùy ý. Phương thức setStatus nhận một int (là status code) như là một tham số. Nếu phản hồi bao gồm một Status

	code đặc biệt và một tài liệu, đảm bảo ta đã gọi setStatus trước khi thực sự trả về bất kỳ nội dung vào với <i>PrintWriter</i>
2	public void sendRedirect(String url) Phương thức này tạo một phản hồi là 302 cùng với một <i>Location</i> header cung cấp URL của tài liệu mới đó
3	public void sendError(int code, String message) Phương thức này gửi một Status code (thường là 404) cùng với một thông báo ngắn mà tự động được định dạng bên trong một tài liệu HTML và được gửi tới Client

Bảng 3: 6 các phương thức để thiết lập HTTP Status code trong Servlet

Ví dụ sau sẽ gửi code lỗi là 407 tới trình duyệt Client và trình duyệt sẽ hiển thị thông báo "Need authentication".

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class showError extends HttpServlet {
    // Method to handle GET method request.
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException
    {
        // Set error code and reason.
        response.sendError(407, "Need authentication!!!" );
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

Bây giờ gọi Servlet trên sẽ hiển thị kết quả sau:

HTTP Status 407 - Need authentication!!!

type Status report
message Need authentication!!!
Mô tả The client must first authenticate itself with the proxy (Need authentication!!!).

Apache Tomcat/9.0

Câu hỏi ôn tập.

1. Servlet là gì và tại sao chúng được sử dụng trong phát triển web Java?
2. Sự khác biệt giữa Servlet và JSP là gì?
3. Làm thế nào Servlet xử lý các yêu cầu HTTP và phản hồi trong ứng dụng web?
4. Servlet Life Cycle là gì? Mô tả các phương thức quan trọng trong Life Cycle.
5. Sử dụng và ý nghĩa của Annotation @WebServlet trong Servlet là gì?
6. Làm thế nào để chuyển dữ liệu giữa các Servlet?
7. Giải thích Session trong Servlet và cách sử dụng nó để duy trì trạng thái trên nhiều yêu cầu.

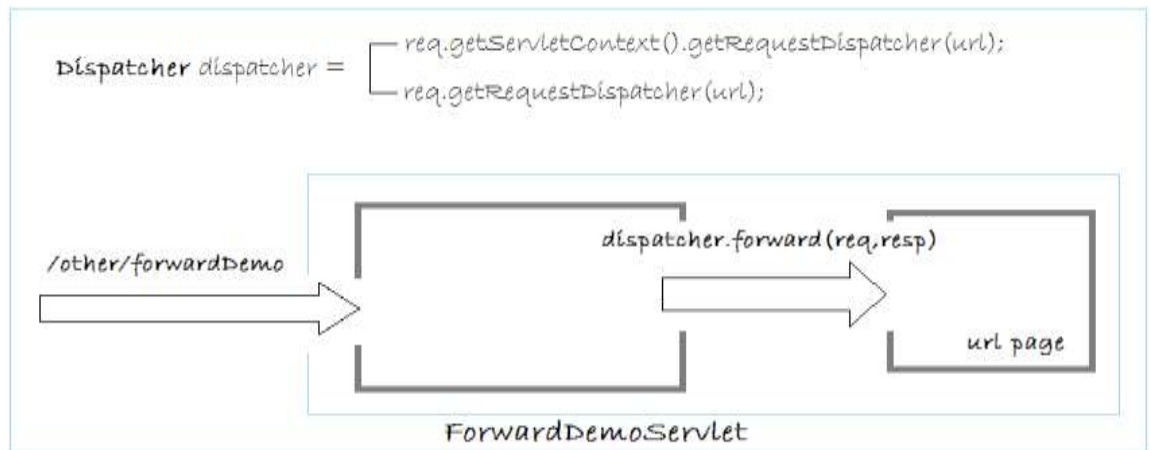
Bài 6: Lập trình Servlet(Tiếp theo)

Nội dung trong bài này chúng ta sẽ tiếp tục tìm hiểu các nội dung quan trọng liên quan đến Servlet như: forward và redirect, Session, cookie, Servlet Filter và cách dùng Filter cũng như Exception trong Servlet.

3.9. Sử dụng forward và redirect trong Servlet

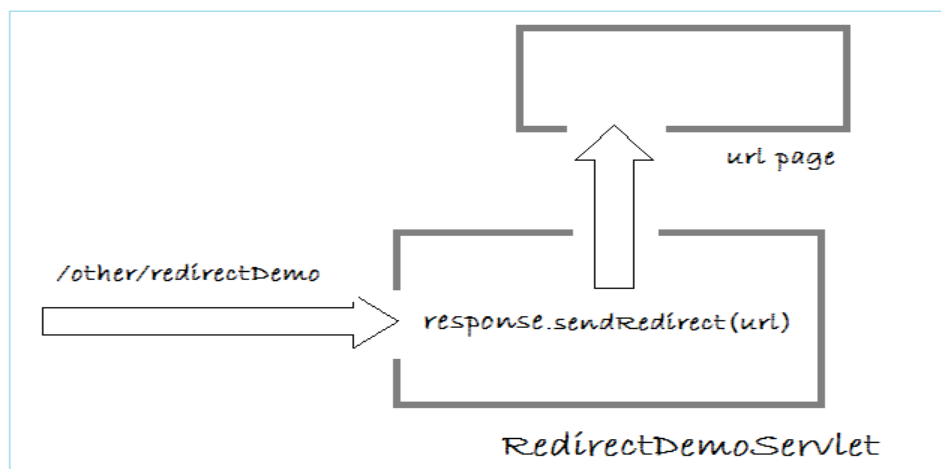
Chuyển tiếp (Forward): Khi một yêu cầu (request) của trình duyệt gửi tới một Servlet, nó có thể chuyển tiếp yêu cầu tới một trang khác (hoặc một servlet khác). Địa chỉ trên trình duyệt của người dùng vẫn là đường dẫn của trang đầu tiên, nhưng nội dung của trang do trang được chuyển tiếp tới tạo ra. Trang được chuyển tiếp tới bắt buộc phải là một trang (hoặc Servlet) nằm trong ứng dụng web.

Với Forward sử dụng `request.setAttribute()` để truyền dữ liệu từ trang 1 sang trang thứ 2.



Hình 3: 6 Forward trong Servlet

Chuyển hướng (Redirect): Khi một yêu cầu (request) từ phía người dùng tới một Servlet (Trang A), servlet này có thể chuyển yêu cầu này tới một trang khác (Trang B), và kết thúc nhiệm vụ của nó. Trang được chuyển hướng tới có thể là trang trong ứng dụng, hoặc có thể là một trang bất kỳ. Địa chỉ trên trình duyệt của người dùng lúc này sẽ hiển thị đường dẫn của trang B.



Hình 3: 7 Redirect trong Servlet

Khác với chuyển tiếp (Forward). Với Redirect ta không thể sử dụng `request.setAttribute(..)` để truyền dữ liệu từ trang A sang trang B.

```

public void HttpServletResponse.sendRedirect(String location)
throws IOException

```

Phương thức này gửi lại phản hồi cho trình duyệt cùng với mã trạng thái và vị trí trang mới. Ta cũng có thể sử dụng `setStatus()` và `setHeader()` các phương pháp với nhau để chuyển hướng trang.

....

```
String site = "http://www.newpage.com" ;
response.setStatus(response.SC_MOVED_TEMPORARILY);
response.setHeader("Location", site);
....
```

3.10. Cookie trong Servlet

Cookie là các tập tin văn bản được lưu trữ trên client. Mục đích của cookie là để theo dõi các thông tin khác nhau. Ví dụ trường hợp remember login.

Có ba bước liên quan đến xác định người dùng cũ quay trở lại hệ thống:

- Tập lệnh của máy chủ gửi một tập hợp các cookie đến trình duyệt. Ví dụ tên, tuổi, hoặc số nhận dạng vv.
- Trình duyệt lưu trữ thông tin này trên máy local để sử dụng trong tương lai.
- Trong lần truy cập tiếp theo, trình duyệt gửi yêu cầu tới web server, nó sẽ gửi những thông tin cookie tới máy chủ và máy chủ sử dụng thông tin đó để xác định người dùng.

Cookie thường được đặt trong HTTP Header (mặc dù JavaScript cũng có thể đặt cookie trực tiếp trên trình duyệt). Một servlet thiết lập một cookie có thể gửi Header trông giống như:

```
HTTP/1.1 200 OK
Date: Fri, 13 Oct 2017 21:03:38 GMT
Server: Apache/1.3.9 (UNIX) PHP/4.0b3
Set-Cookie: name = xyz; expires = Friday, 13-Oct-2017
22:03:38 GMT;
    path = /; domain = abc.vn
Connection: close
Content-Type: text/html
```

Nếu trình duyệt được cấu hình để lưu trữ các cookie, nó sẽ lưu giữ thông tin này cho đến ngày hết hạn. Nếu người dùng chỉ trình duyệt vào bất kỳ trang nào phù hợp với đường dẫn và miền của cookie, nó sẽ gửi lại cookie đến máy chủ. Header của trình duyệt có thể giống như:

```
GET / HTTP/1.0
Connection: Keep-Alive
User-Agent: Mozilla/4.6 (X11; I; Linux 2.2.6-15apmac ppc)
Host: zink.demon.co.uk:1126
Accept: image/gif, */*
Accept-Encoding: gzip
```

Accept-Language: en

Accept-Charset: iso-8859-1,*,utf-8

Một servlet sau đó sẽ có quyền truy cập cookie thông qua yêu cầu của phương thức `request.getCookies()` trả về một mảng các đối tượng `Cookie`.

Các phương thức xử lý cookie trong servlet

Dưới đây là danh sách các phương thức có thể sử dụng trong khi thao tác cookie trong servlet.

STT	Phương thức & Mô tả
1	public void setDomain(String pattern) Phương thức này thiết lập tên miền mà cookie áp dụng, ví dụ như tutorialspoint.com.
2	public String getDomain() Phương thức này lấy tên miền mà cookie áp dụng, ví dụ như tutorialspoint.com.
3	public void setMaxAge(int expiry) Phương thức này đặt khoảng thời gian(tính bằng giây) trước khi cookie hết hạn. Nếu ta không đặt điều này, cookie sẽ chỉ kéo dài cho phiên hiện tại.
4	public int getMaxAge() Phương thức này trả về độ tuổi tối đa của cookie, được chỉ định bằng giây, Theo mặc định, -1 cho biết cookie sẽ tồn tại cho đến khi trình duyệt tắt máy.
5	public String getName() Phương thức này trả về tên của cookie. Không thể thay đổi tên sau khi tạo.
6	public void setValue(String newValue) Phương thức này đặt giá trị kết hợp với cookie
7	String getValue() Phương thức này lấy giá trị kết hợp với cookie.
8	public void setPath(String uri) Phương thức này đặt đường dẫn đến cookie này được áp dụng. Nếu ta không chỉ định đường dẫn, cookie sẽ được trả về cho tất cả các URL trong cùng thư mục với trang hiện tại cũng như tất cả các thư mục con.
9	String getPath() Phương thức này là đường dẫn đến cookie này được áp dụng.
10	public void setSecure(boolean flag) Phương thức này thiết lập giá trị boolean cho biết liệu cookie chỉ nên được gửi qua các kết nối được mã hóa(tức là SSL).
11	public void setComment(String purpose) Phương thức này xác định một comment mô tả mục đích của một cookie. comment này

	hữu ích nếu trình duyệt trình bày cookie cho người dùng.
12	public String getComment() Phương thức này trả về comment mô tả mục đích của cookie này, hoặc không hợp lệ nếu cookie không có comment.

Bảng 3: 7 Các phương thức xử lý cookie trong servlet

Tạo Cookie

Việc tạo cookie trong servlet bao gồm 3 bước sau:

1. Tạo một đối tượng Cookie: gọi constructor Cookie với các tham số tên và giá trị của cookie, cả hai đều có kiểu là String.

```
Cookie cookie = new Cookie("key", "value");
```

Hãy ghi nhớ, tên và giá trị không nên chứa khoảng trắng hoặc bất kỳ ký tự nào sau đây:

```
[ ] ( ) = , " / ? @ : ;
```

2. Thiết định thời gian tồn tại cho Cookie: sử dụng phương thức setMaxAge() để xác định cookie được sống trong thời gian bao lâu (tính bằng giây). Sau đây sẽ thiết lập một cookie sống trong 24 giờ.

```
cookie.setMaxAge(60 * 60 * 24);
```

3. Đính kèm cookie vào HTTP response header: sử dụng phương thức response.addCookie() để thêm các cookie và HTTP response header như sau:

```
response.addCookie(cookie);
```

4. Đọc Cookie trong Servlet

Để đọc cookie, ta cần phải tạo một mảng các đối tượng javax.servlet.http Cookie bằng cách gọi phương thức getCookies() của HttpServletRequest. Sau đó duyệt mảng, và sử dụng phương thức getName() và getValue() để truy cập mỗi cookie và giá trị của nó.

5. Xóa Cookie trong Servlet

Nếu muốn xóa một cookie ta thực hiện ba bước sau:

- 1) Đọc một cookie hiện có và lưu trữ nó trong đối tượng Cookie.
- 2) Đặt tuổi cookie về 0 bằng cách sử dụng phương thức setMaxAge() để xóa cookie.
- 3) Add cookie trở lại response header.

3.11. Session trong Servlet

Một máy chủ web có thể gửi một trường ẩn (Hidden Field) cùng với một session ID duy nhất như sau:

```
<input type="hidden" name="sessionid" value="1234567890">
```

Mục nhập này có nghĩa là, khi form được gửi, tên và giá trị được chỉ định sẽ được tự động đưa vào dữ liệu **GET** hoặc **POST**. Mỗi khi trình duyệt web gửi yêu cầu trở lại, thì giá trị session_id có thể được sử dụng để theo dõi các trình duyệt web khác nhau.

Đây có thể là một cách hiệu quả để theo dõi phiên nhưng khi click vào liên kết siêu văn bản (<A HREF...>) thường không dẫn đến việc gửi form, do đó các trường ẩn cũng không thể hỗ trợ theo dõi phiên.

Sử dụng URL Rewriting

Ta có thể thêm một số dữ liệu bổ sung vào cuối mỗi URL xác định phiên và máy chủ có thể liên kết mã nhận diện phiên với dữ liệu mà nó đã lưu trữ về phiên đó.

Ví dụ: với /test.html?sessionid=1234567890, định danh phiên được đính kèm dưới dạng sessionid = 1234567890, có thể được truy cập tại máy chủ web để xác định client.

Viết lại URL là một cách tốt hơn để duy trì phiên và nó hoạt động ngay cả khi các trình duyệt không hỗ trợ cookie. Hạn chế của việc viết lại URL là ta sẽ phải tự động tạo ra tất cả các URL để chỉ định một session ID, ngay cả trong trường hợp của một trang HTML tĩnh đơn giản.

Đối tượng HttpSession

Ngoài ba cách nêu trên, servlet cung cấp HttpSession interface cung cấp cách để xác định người dùng ghé thăm trang web và lưu trữ thông tin về người dùng đó.

Servlet container sử dụng interface này để tạo ra một session giữa một máy HTTP client và một HTTP server. Session này có thể tồn tại trong một khoảng thời gian nhất định, trên nhiều yêu cầu kết nối hoặc trang của người dùng.

Tạo đối tượng HttpSession bằng cách gọi phương thức getSession() của HttpServletRequest, như sau:

```
HttpSession session = request.getSession();
```

Ta cần gọi request.getSession() trước khi gửi bất kỳ nội dung nào đến client. Dưới đây là tóm tắt các phương thức quan trọng hiện có của đối tượng HttpSession.

STT	Phương thức & Mô tả
1	public Object getAttribute(String name) Phương thức này trả về đối tượng bị ràng buộc với tên được chỉ định trong

	session này, hoặc null nếu không có đối tượng nào bị ràng buộc dưới tên.
2	public Enumeration getAttributeNames() Phương thức này trả về một Enumeration of String các đối tượng có chứa tên của tất cả các đối tượng ràng buộc vào session này.
3	public long getCreationTime() Phương thức này trả về thời gian khi session này được tạo ra, được đo bằng mili giây kể từ nửa đêm ngày 1 tháng 1 năm 1970 GMT.
4	public String getId() Phương thức này trả về một chuỗi chứa mã định danh duy nhất được gán cho session này.
5	public long getLastAccessedTime() Phương thức này trả về thời gian truy cập cuối cùng của session, theo định dạng mili giây kể từ nửa đêm ngày 1 tháng 1 năm 1970 GMT
6	public int getMaxInactiveInterval() Phương thức này trả về khoảng thời gian tối đa (giây), rằng vùng chứa servlet sẽ giữ session mở giữa các lần truy cập của client.
7	public void invalidate() Phương thức này làm mất hiệu lực session này và unbinds bất kỳ đối tượng liên quan đến nó.
8	public boolean isNew() Phương thức này trả về true nếu client chưa biết về session hoặc nếu client không tham gia session làm việc.
9	public void removeAttribute(String name) Phương thức này loại bỏ các đối tượng bị ràng buộc với tên quy định từ session này.
10	public void setAttribute(String name, Object value) Phương thức này liên kết một đối tượng đến session này, sử dụng tên được chỉ định.
11	public void setMaxInactiveInterval(int interval) Phương thức này chỉ định thời gian, tính bằng giây, giữa các yêu cầu của client trước khi bộ chứa servlet sẽ làm mất hiệu lực session này.

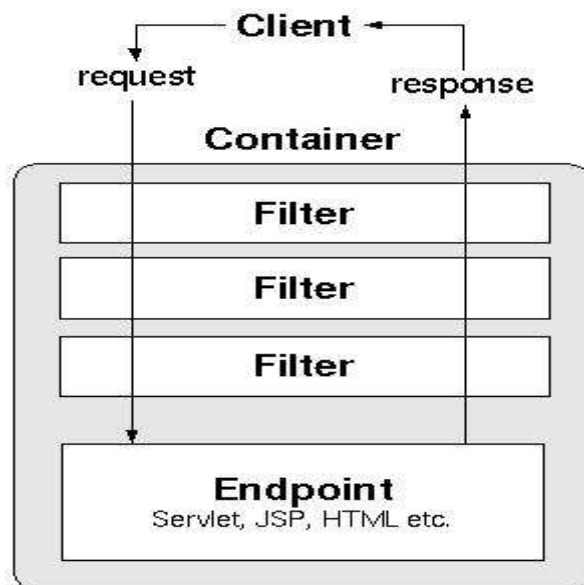
Bảng 3: 8 Các phương thức của đối tượng HttpSession

3.12. Filter trong Servlet

Servlet Filter là các lớp trong Java có thể được sử dụng trong lập trình Servlet cho các mục đích sau:

- Để chặn các yêu cầu từ một Client trước khi chúng truy cập một nguồn tại backend.
- Để thao tác các phản hồi từ Server trước khi chúng được gửi trở lại tới Client.

- Một số loại Filter sau:
 - Authentication Filters – Các Filter xác nhận
 - Data compression Filters – Các Filter nén dữ liệu
 - Encryption Filters – Các Filter mật mã hóa
 - Các Filter mà kích hoạt các sự kiện truy cập nguồn
 - Image Conversion Filters – Các Filter biến đổi hình ảnh
 - Logging and Auditing Filters – Các Filter kiểm tra log và thống kê
 - MIME-TYPE Chain Filters.
 - Tokenizing Filters .
 - XSL/T Filter – Các Filter biến đổi nội dung XML
- Các Filter được triển khai trong **web.xml** file và sau đó map tới hoặc các tên Servlet hoặc các URL pattern.



Hình 3: 8 Servlet Filter

3.12.1. Các phương thức Servlet Filter

Về cơ bản, một Filter là một lớp Java mà triển khai javax.servlet.Filter Interface. Interface này định nghĩa 3 phương thức:

STT	Phương thức và Miêu tả
1	public void doFilter (ServletRequest, ServletResponse, FilterChain) Phương thức này được gọi bởi container mỗi khi một cặp request/response được truyền thông qua chain.
2	public void init(FilterConfig filterConfig)

	Phương thức này được triệu hồi bởi Web container để chỉ một Filter mà đang được đặt bên trong service
3	public void destroy() Phương thức được gọi bởi Web container để chỉ một Filter bị hủy khỏi service

Bảng 3: 9 Các phương thức Servlet Filter

3.12.2. Ví dụ về Servlet Filter

Ví dụ sau về Servlet Filter sẽ in địa chỉ IP và thời gian hiện tại của Client. Ví dụ này sẽ giúp ta hiểu cơ bản về Servlet Filter, nhưng ta cũng có thể viết các ứng dụng Filter phức tạp hơn bởi sử dụng cùng khái niệm này.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Implements Filter class
public class LogFilter implements Filter {
    public void init(FilterConfig config)
        throws ServletException{

        // Get init parameter
        String testParam = config.getInitParameter("test-param");
        //Print the init parameter
        System.out.println("Test Param: " + testParam);
    }

    public void doFilter(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {

        // Get the IP address of client machine.
        String ipAddress = request.getRemoteAddr();

        // Log the IP address and current timestamp.
    }
}
```

```

        System.out.println("IP "+ ipAddress + ", Time "
                                + new Date().toString
        ());

        // Pass request back down the filter chain
        chain.doFilter(request,response);
    }
    public void destroy( ){
        /* Called before the Filter instance is removed
        from service by the web container*/
    }
}

```

Biên dịch LogFilter.java theo cách thông thường và đặt class file trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/classes.

3.12.3. Ánh xạ Servlet Filter trong Web.xml

Các Filter được định nghĩa và sau đó được ánh xạ tới URL hoặc Servlet, theo cách giống như khi Servlet được định nghĩa và sau đó được ánh xạ tới một URL pattern. Tạo entry sau cho Filter trong web.xml file.

```

<filter>
    <filter-name>LogFilter</filter-name>
    <filter-class>LogFilter</filter-class>
    <init-param>
        <param-name>test-param</param-name>
        <param-value>Initialization Paramter</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

Filter trên sẽ áp dụng cho tất cả Servlet bởi vì chúng ta xác định /* trong cấu hình. Ta có thể xác định một Servlet path cụ thể nếu ta chỉ muốn áp dụng Filter trên một số Servlet nào đó.

3.12.4. Sử dụng nhiều Filter trong Servlet

Ứng dụng web có thể định nghĩa một vài Filter cho một mục đích nào đó. Giả sử ta định nghĩa hai Filter là AuthenFilter và LogFilter. Phần còn lại của tiến trình sẽ giống như ví dụ trên, ngoại trừ việc ta cần tạo một ánh xạ khác như dưới đây:

```
<filter>
    <filter-name>LogFilter</filter-name>
    <filter-class>LogFilter</filter-class>
    <init-param>
        <param-name>test-param</param-name>
        <param-value>Initialization Paramter</param-value>
    </init-param>
</filter>

<filter>
    <filter-name>AuthenFilter</filter-name>
    <filter-class>AuthenFilter</filter-class>
    <init-param>
        <param-name>test-param</param-name>
        <param-value>Initialization Paramter</param-value>
    </init-param>
</filter>

<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<filter-mapping>
    <filter-name>AuthenFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

3.12.5. Thứ bậc của các Filter

Bậc của Filter trong web.xml quyết định thứ tự mà Web container áp dụng Filter tới các Servlet. Để đảo ngược thứ tự của Filter, ta chỉ cần đảo ngược các phần tử ánh xạ Filter trong web.xml.

Trong ví dụ trên, LogFilter sẽ được áp dụng đầu tiên và sau đó áp dụng AuthenFilter tới bất kỳ Servlet nào. Nhưng trong ví dụ sau, thứ tự này sẽ bị đảo ngược.

```
<filter-mapping>
    <filter-name>AuthenFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

3.13. Xử lý ngoại lệ (Exception) trong Servlet

Khi một Servlet ném một Exception, thì Web container tìm kiếm các cấu hình (Configuration) trong web.xml mà sử dụng phần tử loại exception này để kết nối với loại exception đã ném.

Ta sẽ phải sử dụng phần tử error.page trong web.xml để xác định sự triệu hồi của các Servlet trong phản hồi tới các exceptions hoặc HTTP status codes cụ thể.

Cấu hình web.xml

Giả sử, ta có một Servlet là ErrorHandler mà sẽ được gọi bất cứ khi nào có bất kỳ exception hoặc error nào được định nghĩa. Sau đây sẽ là entry được tạo trong web.xml

```
<!-- servlet definition -->
<servlet>
    <servlet-name>ErrorHandler</servlet-name>
    <servlet-class>ErrorHandler</servlet-class>
</servlet>
<!-- servlet mappings -->
<servlet-mapping>
    <servlet-name>ErrorHandler</servlet-name>
    <url-pattern>/ErrorHandler</url-pattern>
</servlet-mapping>
<!-- error-code related error pages -->
<error-page>
```

```

        <error-code>404</error-code>
        <location>/ErrorHandler</location>
    </error-page>
    <error-page>
        <error-code>403</error-code>
        <location>/ErrorHandler</location>
    </error-page>
    <!-- exception-type related error pages -->
    <error-page>
        <exception-type>
            javax.servlet.ServletException
        </exception-type>
        <location>/ErrorHandler</location>
    </error-page>
    <error-page>
        <exception-type>java.io.IOException</exception-type>
        <location>/ErrorHandler</location>
    </error-page>

```

Nếu ta muốn có một Error Handler chung cho tất cả Exception, thì ta nên định nghĩa error-page sau thay cho việc định nghĩa từng phần tử error-page riêng biệt cho mỗi exception.

```

    <error-page>
        <exception-type>java.lang.Throwable</exception-type>
        <location>/ErrorHandler</location>
    </error-page>

```

Sau đây là một số điểm đáng chú ý trong web.xml trên để Xử lý Exception:

- ErrorHandler được định nghĩa theo cách thông thường như bất kỳ Servlet nào khác và được định cấu hình bên trong web.xml.
- Nếu có bất kỳ lỗi nào với Status code hoặc 404 (Not Found) hoặc 403 (Forbidden), thì ErrorHandler servlet này sẽ được triệu hồi.
- Nếu ứng dụng web ném hoặc ServletException hoặc IOException, thì Web container triệu hồi ErrorHandler servlet này.
- Ta có thể định nghĩa các Error Handler khác nhau để xử lý các loại Error hoặc Exception khác nhau. Ví dụ trên là một ví dụ chung và hi vọng rằng nó sẽ giải thích cho ta hiểu về khái niệm cơ bản này.

- Các thuộc tính Request: Error/Exception

Bảng dưới liệt kê các thuộc tính request mà một Servlet xử lý lỗi có thể truy cập để phân tích bản chất của Error/Exception:

STT	Thuộc tính và Miêu tả
1	javax.servlet.error.status_code Thuộc tính này cung cấp status code mà có thể được lưu giữ và được phân tích sau khi lưu giữ trong kiểu dữ liệu java.lang.Integer
2	javax.servlet.error.exception_type Thuộc tính này cung cấp thông tin về loại exception mà có thể được lưu giữ và được phân tích sau khi lưu giữ trong một kiểu dữ liệu java.lang.Class
3	javax.servlet.error.message Thuộc tính này cung cấp thông báo lỗi chính xác mà có thể được lưu giữ và được phân tích sau khi lưu giữ trong một kiểu dữ liệu java.lang.String
4	javax.servlet.error.request_uri Thuộc tính này cung cấp thông tin về URL đang triệu hồi tới Servlet và nó có thể được lưu giữ và được phân tích sau khi lưu giữ trong một kiểu dữ liệu java.lang.String
5	javax.servlet.error.exception Thuộc tính này cung cấp thông tin về exception mà có thể được lưu giữ và được phân tích sau khi lưu giữ trong kiểu dữ liệu java.lang.Throwable
6	javax.servlet.error.servlet_name Thuộc tính này cung cấp tên Servlet mà có thể được lưu và được phân tích sau khi lưu giữ trong một kiểu dữ liệu java.lang.String

Bảng 3: 10 Các thuộc tính Error/Exception

Ví dụ về xử lý ngoại lệ Servlet

Sau đây là ví dụ về Servlet mà sẽ được sử dụng như là Error Handler trong bất kỳ trường hợp nào có error hoặc exception xuất hiện với bất kỳ Servlet nào đã định nghĩa. Ví dụ này giúp ta hiểu hơn về Xử lý ngoại lệ trong Servlet, và ta có thể viết các chương trình phức tạp hơn bởi cùng sử dụng khái niệm này.

```
// Import required java libraries
```

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;
// Extend HttpServlet class
public class ErrorHandler extends HttpServlet {
    // Method to handle GET method request.
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // Analyze the servlet exception
        Throwable throwable = (Throwable)
            request.getAttribute("javax.servlet.error.exception");
        Integer statusCode = (Integer)
            request.getAttribute("javax.servlet.error.status_code");
        String servletName = (String)
            request.getAttribute("javax.servlet.error.servlet_name");
;
        if (servletName == null){
            servletName = "Unknown";
        }
        String requestUri = (String)
            request.getAttribute("javax.servlet.error.request_uri");
        if (requestUri == null){
            requestUri = "Unknown";
        }
        // Set response content type
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String title = "Error/Exception Information";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";
    }
}

```

```

        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n" +
            "<body bgcolor=\"#f0f0f0\">\n");

        if (throwable == null && statusCode == null){
            out.println("<h2>Error information is missing</h2>");
            out.println("Please return to the <a href=\"" +
                response.encodeURL("http://localhost:8080/") +
                "\">Home Page</a>.");
        }else if (statusCode != null){
            out.println("The status code : " + statusCode);
        }else{
            out.println("<h2>Error information</h2>");
            out.println("Servlet Name : " + servletName +
                "</br></br>");
            out.println("Exception Type : " +
                throwable.getClass().getName( )
+
                "</br></br>");
            out.println("The request URI: " + requestUri +
                "<br><br>");
            out.println("The exception message: " +
                throwable.getMessage( ));
        }
        out.println("</body>");
        out.println("</html>");
    }

    // Method to handle POST method request.
    public void doPost(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }

```

```
}
```

Biên dịch ErrorHandler.java theo cách thông thường và đặt class file trong <Tomcat-installation-directory>/webapps/ROOT/WEB-INF/classes.

Chúng ta thêm cấu hình sau trong web.xml để xử lý ngoại lệ:

```
<servlet>
    <servlet-name>ErrorHandler</servlet-name>
    <servlet-class>ErrorHandler</servlet-class>
</servlet>
<!-- servlet mappings -->
<servlet-mapping>
    <servlet-name>ErrorHandler</servlet-name>
    <url-pattern>/ErrorHandler</url-pattern>
</servlet-mapping>
<error-page>
    <error-code>404</error-code>
    <location>/ErrorHandler</location>
</error-page>
<error-page>
    <exception-type>java.lang.Throwable</exception-type >
    <location>/ErrorHandler</location>
</error-page>
```

Bây giờ sử dụng một Servlet mà nêu ra một exception hoặc một loại lỗi URL, nó sẽ kích hoạt Web Container để gọi ErrorHandler servlet và hiển thị một thông báo thích hợp như được lập trình. Ví dụ, nếu ta soạn một URL lỗi, nó sẽ cho thông báo sau:

```
The status code : 404
```

Câu hỏi ôn tập.

1. Giải thích Session trong Servlet và cách sử dụng nó để duy trì trạng thái trên nhiều yêu cầu.
2. Tại sao nên sử dụng Servlet Filter và ví dụ về một tình huống sử dụng nó.
3. Filter làm việc như thế nào trong mô hình xử lý yêu cầu và phản hồi trong Servlet?
4. Sự khác biệt giữa Filter và Servlet là gì? Khi nào nên sử dụng Filter?
5. Làm thế nào để xác định thứ tự thực hiện của nhiều Filter trong một ứng dụng web Servlet?

BÀI TẬP THỰC HÀNH

Bài 1.

Tạo một Servlet đơn giản có thể xử lý yêu cầu GET và trả về một trang HTML cơ bản với thông điệp chào hỏi.

Bài 2.

Tạo một ứng dụng sử dụng Servlet để đếm số lần một người dùng truy cập trang web. Sử dụng phiên để duy trì số lần truy cập này và hiển thị nó trên trang web.

Bài 3.

Tạo một Servlet kết nối đến cơ sở dữ liệu (sử dụng JDBC) và hiển thị dữ liệu từ một bảng cụ thể (ví dụ: danh sách sản phẩm, người dùng) lên trang web.

Bài 4.

Tạo một trang web với một biểu mẫu đơn giản (ví dụ: nhập tên và email). Tạo một Servlet để xử lý yêu cầu POST từ biểu mẫu và hiển thị thông tin đã nhập lên trang web.

Bài 5.

Để hát triển một ứng dụng quản lý sách trực tuyến. Mỗi khi người dùng thêm một cuốn sách mới, họ cần nhập thông tin về tên sách và tác giả. Tuy nhiên, để đảm bảo an toàn và chất lượng dữ liệu, ta cần thêm một tính năng Filter để kiểm tra và chuyển đổi tên sách và tên tác giả thành định dạng chuẩn.

Yêu cầu:

Tạo một Servlet có tên là AddBookServlet để xử lý việc thêm sách mới vào hệ thống.

Tạo một Filter có tên là InputValidationFilter để kiểm tra và chuyển đổi dữ liệu nhập vào từ người dùng.

Filter này sẽ kiểm tra các tham số bookTitle và authorName được gửi từ AddBookServlet.

Nếu các tham số không đáp ứng định dạng chuẩn, Filter sẽ chuyển đổi chúng để đảm bảo rằng tên sách và tên tác giả được lưu trữ đúng cách trong cơ sở dữ liệu.

Sau đó, AddBookServlet sẽ tiếp tục xử lý và thêm sách vào hệ thống.

